COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S4	**SECTION:**	CPE21S4
DATE PERFORMED:	21/10/2026	**DATE SUBMITTED:**	21//10/2026
NAME:	Jullian Edrain R. Sevilla Angel Mae C. Ramirez	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Merge Sort	O(n log n)	O(n log n)	Merge Sort divides the array into smaller parts and merges them in sorted order. Its time complexity remains O(n log n) regardless of the initial arrangement of the elements.
Quick Sort	O(n log n)	O(n^2)	While Quick Sort performs efficiently when the pivot divides the array into reasonably balanced partitions. Its performance becomes slower when the partitions are highly unbalanced.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Merge Sort	Quick Sort
Best Case	`Best Case Time:  5.6722 ms`	`Best Case Time:  1.7793 ms`
Worst Case	`Worst Case Time: 5.3954 ms`	`Worst Case Time: 745.265 ms`
Analysis:	Merge Sort is very consistent. Both best and worst cases took around 5ms because it always splits and merges data the same way regardless of the initial order.	Quick Sort is super fast in the best case (1.7ms), but slows down massively in the worst case (745ms) because a bad pivot selection on sorted data forces too many comparisons.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A. Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

Pseudocode Analysis

Binary Search is faster than Linear Search when working with a large amount of sorted data. It works by checking the middle element and deciding whether to continue searching on the left or right side. This process cuts the search area in half each time, making it faster to find the target. On the other hand, Linear Search checks each element one by one until the target is found or the list ends. It can take longer when there are many elements, but it is useful for smaller lists or when the data is not sorted.

Time and Space Complexity

Linear Search checks the elements one at a time, so the time it takes depends on how many elements are in the list. As the input size increases, the search may take longer. Its time complexity is O(n). Binary Search is more efficient because it cuts the search area in half during every step, giving it a time complexity of O(log n). Both algorithms have a space complexity of O(1) when implemented iteratively because they only use a fixed amount of extra memory regardless of the input size. Overall, Binary Search is a better choice for searching large, sorted lists, while Linear Search is still useful when dealing with small or unsorted lists.

Input size (N)	Elapsed Time for Binary Search	Elapsed Time for Linear Search
1,000	Worst Case Binary Search Algorithm's Time: 200 nanoseconds	Worst Case Linear Search Algorithm's Time: 6300 nanoseconds
10,000	Worst Case Binary Search Algorithm's Time: 300 nanoseconds	Worst Case Linear Search Algorithm's Time: 84200 nanoseconds
100,0000	Worst Case Binary Search Algorithm's Time: 300 nanoseconds	Worst Case Linear Search Algorithm's Time: 635400 nanoseconds

Analysis:

Sevilla: Linear Search gets significantly slower as N grows (jumping from 6,300 ns to 635,400 ns) because it checks elements one by one, showing linear O(N) complexity. On the other hand, Binary Search stays almost instant (200 ns to 300 ns) even at N = 100,000 because it continuously cuts the search space in half (O(\log N)). This proves that Binary Search is far faster and more efficient for large sorted datasets.

Ramirez: Based on the results, Linear Search takes longer as the input size increases. When the number of elements increases from 1,000 to 10,000 and then to 100,000, its execution time also increases significantly because it checks each element one by one. In comparison, Binary Search shows a smaller increase in execution time even when the input size becomes ten times larger. This is because it divides the search area in half during each step, allowing it to search through large amounts of data more efficiently. These results support the theoretical analysis that Linear Search has a time complexity of O(n), while Binary Search has a time complexity of O(log n).

Conclusion:
Sevilla: In this activity, I relearned how code speed actually works in real life by testing sorting and searching algorithms. Merge sort was super reliable and always took around 5 ms no matter what, while Quick sort was faster at 1.7 ms normally but slowed down badly to 74.5 ms when the data was already sorted. For searching, Linear Search got really slow, up to 635,400 ns, because it checks items one by one, while Binary Search stayed super fast at around 300 ns even for 100,000 items because it keeps cutting the list in half. Overall, our code results matched our theory perfectly, and next time we just need to improve Quick Sort so it doesn't lag on sorted lists.
Ramirez: In this activity, I learned about algorithm complexity and how it affects a program's performance. I learned that time complexity measures how the running time changes as the input size increases, while space complexity refers to the extra memory an algorithm uses. I also compared Merge Sort and Quick Sort and learned that they sort data in different ways. Merge Sort has more consistent performance, while Quick Sort's performance depends on how the pivot divides the data. For the supplementary activity, I compared Binary Search and Linear Search through theoretical and experimental analysis. Based on the results, Binary Search was faster when searching through larger amounts of sorted data because it reduces the search area by half in each step. Meanwhile, Linear Search checks each element one by one, which can take longer as the input size increases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Did not replace the original thinking**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.